A small Swedish language model for a bathroom webshop

Jasmin Jakupovic · Understanding and Building Large Language Models, LiU VT26 · `github.com/JJakupovic/badrum-llm`

Setup

I am building a webshop for bathroom products in Medusa and used it as the test environment for this course. Medusa puts variants and their option values directly under the product, so I generated a synthetic Swedish catalogue with the same shape: 400 products, 1945 variants, 982 documents, 330k training tokens, all deterministic from a seed. From it I generated 2800 instruction pairs across ten task types (price, variant count, stock, SKU, add-to-cart and so on), each carrying a gold answer computed from the catalogue rather than written by hand.

The model is a decoder-only transformer written from scratch following Raschka ch. 3-4; only `nn.Linear`, `nn.Embedding` and the optimiser come from PyTorch. I pretrained on the catalogue with a byte-level tokenizer, instruction-tuned with the loss masked to answer tokens only, and evaluated on 420 held-out pairs from 60 products excluded from pretraining as well as fine-tuning. Because every gold answer is computed, scoring is exact match, set F1 or argument equality, with no judge model. Two baselines run through the same scorer: **majority** answers the most common gold value per task, **retrieval** answers with the nearest training question's gold answer.

Results

	params	pretrain perplexity	task accuracy (macro)
tiny	10.8M	1.46	**0.58**
small	25.5M	1.27	**0.59**
tiny, no pretraining	10.8M	n/a	0.34
majority baseline			0.22
retrieval baseline			0.36

Experiment 1, model scale. From 0.1M to 10.8M parameters everything changes. From 10.8M to 25.5M, perplexity keeps improving while task accuracy does not, and the per-task differences run in both directions without clearing noise. Perplexity stops predicting task performance well before it stops improving.

Experiment 2, does pretraining help? Same architecture, data, schedule and seed, differing only in initialisation. The pretrained model reaches 0.58 against 0.34 from random init, and the control falls below the retrieval floor of 0.36. The gap is not spread evenly: on fact-dependent tasks the two are level, while SKU goes 0.86 against 0.00 and add-to-cart 0.92 against 0.00. Pretraining did not broadly improve the model. It made one compositional character-level rule learnable at all.

What the model learned, and what it could not

The evaluation divides cleanly in two. Three tasks are answerable from the question itself: the SKU is a fixed function of the product noun, model name and option values, and the category follows from the noun. The model scores 1.00 on category, 0.92 on add-to-cart and 0.86 on SKU, against retrieval floors of 0.87, 0.00 and 0.00. The other seven need facts about a product it never saw, and there it sits at or below the floors: price and cheapest at 0.00, variant count at 0.30 against the 0.34 a constant answer achieves, option check at 0.64 against retrieval's 0.72.

So the model learned the form of the answers and the deterministic rules, and learned nothing about facts it could not derive. It also cannot count: sampling produced *"finns i fem varianter"* above a list of four. For the shop I am building this settles the architecture. Prices, stock and variant counts have to come from a query with the model choosing the arguments, not from weights.

A note on method

Six bugs turned up during the project and every one of them made a result look better rather than worse: a leaking causal mask, a validation split by token position, an off-by-one loss mask, an instruction split by pair instead of by product, a task whose gold answer was always *"Nej"* and on which a constant answer therefore scored 100%, and a containment metric that credits a hedged answer. None crash, and four make a curve look better than the truth. Each countermeasure is a test that I then broke on purpose to confirm it could fail.

Limitations

The corpus is entirely machine-generated and its variety is bounded by my template file, not by the product count. A validation perplexity of 1.27 says the text is close to memorisable, which is lecture 3's model-collapse argument with a number attached. Forty-two validation pairs per task means differences under roughly fifteen points are noise, covering all of small against tiny. Decoding is greedy throughout, so I have not measured how much of the gap survives sampling. The macro average is a weak summary: at 0.59 it largely measures how many of my ten tasks are rule-derivable, and the per-task split is the real result.

I used an AI assistant for design discussion and for drafting code and documentation, which I then reviewed, ran and corrected. The scope decisions, the experiments and the analysis above are mine. The repository README records this in more detail.